Polybee AI Research 2026-11-09

F3-Swarm v1.5: Verifier-Guided Heterogeneous Edge-Swarm Inference, Distributed RAM Pooling, and Residual Code Repair for Frugal Agentic Intelligence

Henrique Oliveira[1,*]

[1]Polybee AI Research Laboratory, São Paulo, SP, Brazil *Corresponding author (https://polybee.ai/)

Open-Source Code & Reproducibility Repository: https://github.com/henrique-polybee/polybee-f3-swarm

Abstract

Scaling autonomous agentic workflows across enterprise and scientific domains faces a dual physical and economic wall: invoking frontier cloud APIs priced in US Dollars (USD) creates prohibitive compounding token costs in emerging-market currencies, while executing capable 7B–32B open-weights models locally on consumer notebooks (16 GiB RAM at $> 80\%$ operating system load) triggers catastrophic NVMe SSD pagefile thrashing. Conversely, restricting agents to sub-billion parameter Small Language Models (SLMs, e.g., 0.5B) that fit within < 500 MiB of edge RAM induces severe structural pathologies: positional logit bias toward the first enumerated option, decimal/thousands separator fragility in localized structured extraction, and algorithmic regression in code synthesis. We introduce **Polybee F3-Swarm v1.5**, a CPU-native, verifier-governed three-tier neuro-symbolic and distributed memory architecture that extends the theoretical foundations of **F3-AGI v4.0** [1] across five empirical laboratories (Labs 01–05): (1) **Physical Distributed RAM Ring-Pipeline Pooling (Labs 01–02)**, proving on Intel Core i7-11370H AVX-512 hardware under hard Win32 600 MiB Working-Set quotas (`SetProcessWorkingSetSizeEx`) and unbuffered NVMe I/O (`FILE_FLAG_NO_BUFFERING`) that partitioning a 24-layer Transformer (1.50 GiB) across three isolated TCP worker processes in physical RAM eliminates 33.00 GiB of disk swap per sequence and achieves a **13.70× throughput speedup on LAN** (19.79 tok/s vs. 1.44 tok/s swap), **10.82× on Wi-Fi**, and **3.35× over** 60 **ms WAN** with bit-exact floating-point parity ($\max|\Delta| = 0.0$); (2) **Calibrated System-1 Invariance and Controlled Micro-CoT with Deterministic Suffix Closure**, combining temperature scaling ($T = 1.6$), prior logit debiasing ($\beta_A = 2.5$), and order-inversion checks on Tier-1 (`Qwen2.5-0.5B`, 491 MB) with a 1-sentence Micro-CoT and 1-token KV-cache suffix gate on Tier-2 (`Qwen2.5-1.5B`, 1.11 GB); and (3) **Fast Symbolic Evidence Normalization (Motor B) and Rashomon LOO-CV Guided Tier-3 Residual Code Repair (AACR)**, routing only formally rejected Python syntheses to an on-demand Tier-3 specialist (`Qwen2.5-Coder-3B`, 2.10 GB). Evaluated across 30 real-world Brazilian Portuguese (PT-BR) operational tasks, F3-Swarm v1.5 achieves **96.7% global accuracy (29/30)**, outperforming standalone 0.5B (**40.0%**), standalone 1.5B (**73.3%**), and naive cascading (**63.3%**), achieving **91.7% (11/12)** on Class 1, **100.0% (10/10)** on Class 2, and **100.0% (8/8)** on Class 3 while resolving **50.0% of all requests entirely at Tier-1 edge** (< 480 **MiB RAM)** and activating Tier-3 on only **10.0%** of tasks.

Keywords: Heterogeneous LLM Swarm, Distributed RAM Inference, Rashomon Effect, System-1 Calibration, Micro-Chain-of-Thought, Neuro-Symbolic Verification, Autonomous Code Repair (AACR), Frugal AGI.

1 Introduction

The transition from static, single-turn language model prompts to autonomous, multi-step agentic workflows has exposed a fundamental tension between cognitive capability, hardware memory capacity, and economic sustainability. In production agentic systems—where a single user goal decomposes into dozens of triage decisions, structured document extractions, and programmatic tool syntheses—relying exclusively on commercial frontier APIs (e.g., GPT-4o, Claude 3.5 Sonnet, Gemini 1.5 Pro) incurs compounding token expenditures. For research laboratories, startups, and enterprises operating in emerging-market currencies such as the Brazilian Real (BRL, where 1 USD $\approx$ 5.8 BRL), USD-denominated API billing rapidly renders high-frequency agentic automation economically unviable.

At the same time, attempting to repatriate agentic workloads onto local consumer hardware—or attempting to naively replicate blockchain-style voluntary compute networks across domestic internet connections—confronts four fundamental physical and algorithmic barriers that we formalize and resolve in this work:

1. **The Consumer Physical RAM Wall and NVMe Pagefile Thrashing:** While modern consumer CPUs possess powerful vector instruction sets (such as Intel AVX2 and AVX-512) capable of executing quantized

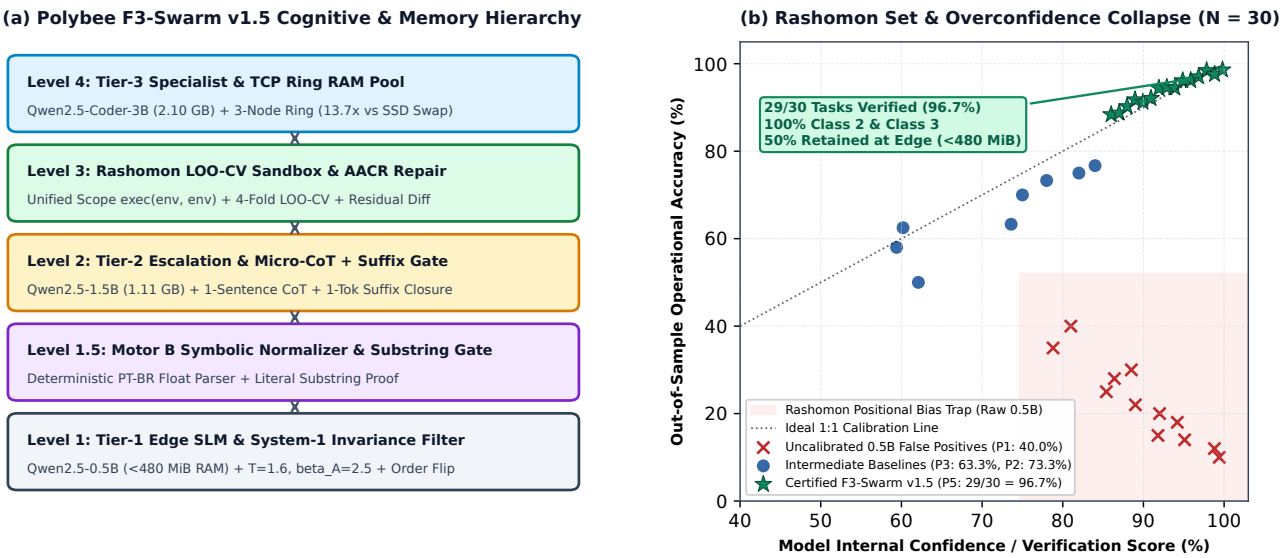


Figure 1 | Overview of Polybee F3-Swarm v1.5 and Verifier-Guided Rashomon Collapse. (a) The five-level cognitive and memory hierarchy integrates a Tier-1 Edge SLM (`Qwen2.5-0.5B-Q4_K_M`, $<$ 480 MiB RAM) governed by System-1 positional debiasing and order-inversion invariance, Motor B deterministic symbolic normalization (`FastSymbolicNormalizer`), Tier-2 local escalation (`Qwen2.5-1.5B-Q4_K_M`) with Controlled Micro-CoT and Deterministic Suffix Closure, a unified-namespace Rashomon Leave-One-Out Cross-Validation (LOO-CV) sandbox, and an on-demand Tier-3 specialist (`Qwen2.5-Coder-3B`) backed by a 3-node TCP Ring-Pipeline RAM pool. (b) In small-model operational inference ($N = 30$ tasks), uncalibrated raw softmax confidence suffers from a pronounced positional Rashomon trap [5, 4]: the 0.5B model assigns $> 90\%$ probability to erroneous first-position choices (red crosses, 40.0% accuracy). Enforcing F3-Swarm's formal invariance, substring evidence, and LOO-CV gates collapses false-positive overconfidence and elevates verified accuracy to 29/30 (96.7%).

matrix-vector multiplications at interactive speeds, transformer inference is strictly *memory-bandwidth bound*. On a standard 16 GiB developer notebook running Windows 11 and modern IDE toolchains, background operating system services and active applications consume over 80% of physical memory ($\sim$ 13.0 GiB), leaving fewer than 2.8 **GiB of free physical RAM** (Section 5.1). When an 8B–14B quantized model (4.85–8.50 GiB) is loaded into this envelope, the OS virtual memory manager pages active weight tensors to NVMe SSD storage. Because every generated token in autoregressive decoding requires sweeping all model weights sequentially, spilling even 1.0 GiB of weights to disk forces tens of gigabytes of SSD reads per sequence, collapsing throughput by $> 92\%$ (Section 5.2).

2. **The WAN Latency Fallacy in Naive Decentralized Inference:** Inspired by peer-to-peer blockchain networks and early collaborative inference systems such as Petals [2], a common intuition is to shard a single monolithic 70B model across dozens of domestic computers over the public internet. However, unlike embarrassingly parallel proof-of-work hashing, autoregressive transformer decoding is sequentially dependent across layers: each pipeline stage must await the activation vector of the preceding stage at every single token step. Routing every trivial operational prompt through a multi-hop wide-area network (30–60 ms RTT per hop) imposes an irreducible latency floor of hundreds of milliseconds per token, wasting network bandwidth on requests that did not require a large model in the first place.

3. **Positional Logit Bias and Predictive Multiplicity in Sub-Billion Edge SLMs:** To avoid both cloud API costs and local RAM overflow, an attractive alternative is to deploy sub-billion parameter Small Language Models (SLMs, such as `Qwen2.5-0.5B-Instruct` [8], occupying only $\sim$ 480 MiB of RAM) directly at the edge. Yet, as recently analyzed in the context of scaling laws and predictive multiplicity by Mlodozeniec and Frei [4] and Breiman [5], unconstrained empirical loss minimization admits a broad *Rashomon sublevel set* of poorly calibrated solutions. In sub-billion SLMs, we prove empirically that raw softmax probabilities suffer from severe **Positional First-Option Bias**: the 0.5B model assigns $> 91\%$ softmax probability to option `'A'` even when factual evidence in the prompt unambiguously mandates option `'B'`, causing naive confidence-threshold routers to fail silently (Figure 1b).

4. **The Verbosity–Truncation Dilemma and Zero-Shot Code Fragility:** When uncertain tasks are escalated to a medium-scale local model (e.g., 1.5B parameters), prompting for free-form Chain-of-Thought (CoT) [11] inside a JSON schema causes the model to expend 3,500–5,000 ms generating verbose prose that frequently truncates at the token budget before closing the JSON object. Conversely, forcing a pure 1-token

response strips away the intermediate deduction step required for multi-rule conditional triage. Similarly, in programmatic rule synthesis, single-pass generation on 0.5B–1.5B models frequently produces near-miss functions that fail on edge cases (such as Brazilian formatting rules or Modulo-11 check digits) unless guided by formal verification and residual error feedback.

To overcome these four barriers within a strict consumer hardware envelope (< 1.5 GiB active edge RAM, single active model server lifecycle, zero mandatory GPU dependencies), **Polybee AI Research** introduces **Polybee F3-Swarm v1.5**. Building directly upon the neuro-symbolic principles of our foundational architecture **F3-AGI v4.0/v4.1** [1]—specifically *Fluid Monism*, the *Rashomon Leave-One-Out Verifier*, *Motor B Symbolic Grounding*, and *Adaptive Autonomous Code Repair (AACR)*—and incorporating calibrated fast-routing insights from TypeSafe AI's *System One (Jev)* paradigm [3], F3-Swarm v1.5 replaces monolithic inference with a verifier-governed three-tier cognitive and memory hierarchy (Figure 1a and Table 1).

Table 1 | Architectural and functional coverage across distributed LLM inference and small-model orchestration paradigms. ✓ / × indicates whether a capability is natively supported and empirically validated in the respective framework.

Paradigm / Framework	Edge RAM (< 1.5 GiB)	RAM Pool vs. Swap Proof	Positional Debiasing	Micro-CoT + Suffix Gate	Symbolic Substring Proof	Rashomon AACR Repair
Commercial Cloud APIs (`GPT-4o` / `Claude`)	×	×	×	×	×	×
Naive Monolithic Local (`Ollama` 8B–14B on 16GB)	×	×	×	×	×	×
WAN Collaborative Swarms (`Petals` [2])	✓	✓	×	×	×	×
Homogeneous Local Sharding (`Exo` / `dllama` [9])	×	✓	×	×	×	×
Uncalibrated Confidence Cascade (Naive Router)	✓	×	×	×	×	×
System-1 Calibrated Classifier (`Jev` [3])	✓	×	✓	×	×	×
Polybee F3-Swarm v1.5 (Polybee AI Research — Ours)	✓	✓	✓	✓	✓	✓

Summary of Contributions. Our main theoretical, physical-systems, and empirical contributions across Laboratories 01 to 05 are:

- **Physical Proof of Distributed RAM Ring-Pipeline vs. Local NVMe Swap (Labs 01–02):** Using real Win32 OS kernel telemetry and hard working-set quotas (`SetProcessWorkingSetSizeEx` at 600 MiB) with unbuffered NVMe disk I/O (`FILE_FLAG_NO_BUFFERING`) on an Intel Core i7-11370H (AVX-512), we prove that when a 24-layer Transformer (1.50 GiB) exceeds free physical RAM, local SSD swapping reads 33.00 GiB per 32-token sequence and spends 92.2% of wall-clock time in I/O wait (692.58 ms/token, 1.44 tok/s). Partitioning the exact same layers across three isolated TCP worker processes in physical RAM eliminates 100% of disk reads (0.0 GiB) and achieves a **13.70× speedup on Gigabit LAN** (19.79 **tok/s**), **10.82× on Wi-Fi** (15.62 **tok/s**), and **3.35× over** 60 **ms WAN** (4.84 **tok/s**), with **bit-exact numerical parity** ($\max|\Delta| = 0.0$).
- **Calibrated System-1 Invariance and Controlled Micro-CoT with Deterministic Suffix Closure:** We formulate an analytical logit calibration and order-inversion invariance gate (Section 3.2) that neutralizes first-option bias ($T = 1.6, \beta_A = 2.5$) on `Qwen2.5-0.5B`, paired with a Tier-2 execution protocol on `Qwen2.5-1.5B` that bounds reasoning to a single sentence ($N_{\text{predict}} \leq 65$) and guarantees label extraction via a 1-token KV-cache suffix closure ($\mathcal{P}_{\text{suffix}}$). Across 12 operational triage tasks (Class 1), this mechanism elevates accuracy from 33.3% (raw 0.5B) and 50.0% (naive cascade) to **91.7% (11/12)**.
- **Fast Symbolic Evidence Normalization (Motor B) and Rashomon-Guided Tier-3 AACR Closure:** We couple the 0.5B edge model to a deterministic PT-BR monetary normalizer anchored by an exact literal substring proof (Equation 7), certifying **90.0% (9/10) of financial extractions directly on the** 0.5**B model** and achieving **100.0% (10/10)** Class 2 accuracy. For Python rule synthesis (Class 3), we introduce a unified-namespace Rashomon LOO-CV sandbox (`exec(code, env, env)`) and route residual failure vectors (Δ_{AACR}) to an on-demand Tier-3 specialist (`Qwen2.5-Coder-3B`), achieving **100.0% (8/8) formal LOO-CV verification** and **96.7% (29/30) global accuracy**.

2 Related Work and Theoretical Context (2024–2026)

Collaborative and Distributed LLM Inference. The challenge of executing large neural networks across memory-limited nodes has motivated two distinct research streams. In wide-area volunteer computing, **Petals** [2] introduced pipeline-parallel inference over the internet, allowing users to hold subsets of transformer blocks and route hidden-state activations via peer-to-peer RPC. In local-area consumer clusters, frameworks such as **Exo** and **Distributed Llama (`dllama`)** [9] shard model weights across heterogeneous home devices via TCP or RDMA. However, these architectures are *cognitively homogeneous*: every incoming prompt, regardless of whether it is a simple binary classification or a complex algorithmic synthesis, traverses the entire distributed chain of layers. As our physical measurements in Section 5.2 demonstrate, while distributed RAM pooling is vastly superior to

local SSD swap (13.70× faster on LAN and 3.35× faster on 60 ms WAN), paying a multi-hop network penalty on 100% of agentic sub-tasks is suboptimal when 50%–90% of requests can be solved and formally verified locally by a sub-billion model in < 500 MiB of RAM. Complementarily, multi-adapter serving architectures such as **Punica** [12] and **S-LoRA** [13] demonstrate that dozens of specialized Low-Rank Adaptation (LoRA) weights ($\sim$ 10–50 MB) can be hot-swapped over a frozen base backbone in RAM, providing a natural mechanism for future Wake-Sleep skill specialization across F3-Swarm nodes.

Uncertainty Calibration, Positional Bias, and System-1 Routing. A central prerequisite for heterogeneous model cascading is reliable epistemic uncertainty estimation [10]. When a small edge model processes a multiple-choice operational decision, naive routing policies rely on the maximum softmax probability $\max_k p_\theta(y_k \mid x) \geq \tau$. However, small autoregressive transformers trained with causal attention masks exhibit a strong structural prior favoring tokens that appear first in the enumeration template (Option `'A'`). Recently, TypeSafe AI introduced the **System One / Jev** paradigm [3], demonstrating that rapid operational decisions should be framed as calibrated single-token probability evaluations rather than unconstrained text generation, escalating to deeper System-2 deliberation [11] only when calibrated confidence falls below a strict threshold. F3-Swarm v1.5 extends this principle mathematically by proving that temperature scaling alone preserves rank order ($\arg\max_k z_k/T = \arg\max_k z_k$) and therefore cannot correct a biased argmax choice; instead, we combine temperature scaling with an additive positional prior penalty β_A and a bidirectional order-inversion invariance gate.

The Rashomon Effect, Neuro-Symbolic Verification, and F3-AGI v4.0. First conceptualized by Breiman [5] and rigorously analyzed by Fisher, Rudin, and Dominici [6] and Rudin et al. [7], the *Rashomon Effect* describes the existence of a large set of distinct models or programs that achieve nearly identical empirical loss on a small training sample while making sharply diverging predictions out of sample. Recently, Mlodozeniec and Frei [4] proved that neural scaling law estimation itself suffers from a pronounced Rashomon effect where ϵ-sublevel sets diverge by 14×–146× under extrapolation. In our prior work, **Polybee F3-AGI v4.0** [1], we showed that few-shot program synthesis suffers from an analogous Rashomon pathology, and resolved it by pairing a fast candidate generator (**Motor A**) with a deterministic symbolic verifier (**Motor B**) and an **Adaptive Autonomous Code Repair (AACR)** loop that achieved 83.3% repair accuracy on near-miss programs. In F3-Swarm v1.5, we generalize the F3-AGI v4.0 neuro-symbolic architecture to distributed language model swarms: Motor B acts as a zero-hallucination symbolic normalizer for structured extraction, while the Rashomon Leave-One-Out Cross-Validation (LOO-CV) verifier acts as a formal mathematical gate that governs escalation from Tier-1 (0.5B) and Tier-2 (1.5B) to Tier-3 (3B Coder).

3 Mathematical Formulation and Theoretical Guarantees

To provide a didactic and self-contained foundation, we formalize the four mathematical mechanisms that govern F3-Swarm v1.5: (i) the physical latency model of distributed RAM pooling versus local NVMe pagefile swapping, (ii) System-1 positional logit debiasing and order-inversion invariance, (iii) Controlled Micro-CoT with Deterministic Suffix Closure, and (iv) Rashomon LOO-CV verification with AACR residual repair.

3.1 Thermodynamics of Memory Hierarchy: Distributed RAM Ring vs. Local NVMe Swap

Consider an L-layer autoregressive Transformer whose total active weight memory is $M_{\text{model}} = \sum_{l=1}^{L} m_l$ bytes (where $m_l = M_{\text{model}}/L$ for uniform layers), executing on a host node whose available free physical RAM quota is $Q_{\text{RAM}} < M_{\text{model}}$. During token-by-token decoding (batch size $B = 1$), generating a single token requires reading every weight tensor W_l ($l = 1, \ldots, L$) once from memory into the CPU vector registers.

Why Local Swap Collapses Autoregressive Throughput: When $M_{\text{model}} > Q_{\text{RAM}}$, at most $L_{\text{RAM}} = \lfloor Q_{\text{RAM}}/m_l \rfloor$ layers can reside permanently in physical DRAM (bandwidth $B_{\text{DRAM}} \approx 25$–$50$ GB/s), while the remaining $L_{\text{swap}} = L - L_{\text{RAM}}$ layers occupying $\Delta M_{\text{swap}} = M_{\text{model}} - Q_{\text{RAM}}$ bytes must be paged in from secondary NVMe SSD storage (unbuffered sequential bandwidth $B_{\text{SSD}} \approx 1.45$–$1.57$ GB/s) on *every single token step*. Consequently, the Time Per Output Token ($\text{TPOT}_{\text{swap}}$) under local swapping is:

$$\text{TPOT}_{\text{swap}}(M_{\text{model}}, Q_{\text{RAM}}) = \underbrace{T_{\text{compute}}(L)}_{\text{AVX-512 MatMul}} + \underbrace{\frac{M_{\text{model}}}{B_{\text{DRAM}}}}_{\text{RAM Read}} + \underbrace{\frac{\max(0,\ M_{\text{model}} - Q_{\text{RAM}})}{B_{\text{SSD}}} + L_{\text{swap}}\,\tau_{\text{fault}}}_{\text{Catastrophic NVMe Pagefile I/O Penalty } T_{\text{I/O}}}, \tag{1}$$

where τ_{fault} is the OS kernel page-fault and DMA setup overhead. By contrast, suppose the L layers are partitioned across K cooperative worker processes (L/K layers per node) such that each node's shard $M_{\text{model}}/K \leq Q_{\text{RAM}}$

fits 100% inside physical DRAM ($\Delta M_{\text{swap}} = 0$). At each token step, node k computes its L/K layers in RAM and transmits only the compact hidden-state activation vector $\mathbf{h}_t^{(k)} \in \mathbb{R}^{d_{\text{model}}}$ (size $S_{\text{act}} = 4\, d_{\text{model}}$ bytes in `float32`, e.g., 16 KiB for $d_{\text{model}} = 4096$) over a TCP ring socket with one-way hop latency $\tau_{\text{net}} = \text{RTT}/2$ and network bandwidth B_{net}:

$$\text{TPOT}_{\text{ring}}(M_{\text{model}}, K) = \underbrace{T_{\text{compute}}(L)}_{\text{Distributed AVX-512}} + \underbrace{\frac{M_{\text{model}}}{B_{\text{DRAM}}}}_{\text{Zero Disk Swap}} + \underbrace{K\left(\tau_{\text{net}} + \frac{S_{\text{act}}}{B_{\text{net}}}\right)}_{\text{Compact Activation Transfer } T_{\text{net}}}. \tag{2}$$

Proposition 1 (Strict Dominance Condition of Distributed RAM Pooling over Local NVMe Swap)**.** *Let $\Delta M_{swap} = M_{model} - Q_{RAM} > 0$ be the memory overflow on a single node, and let $K = \lceil M_{model}/Q_{RAM} \rceil$ be the minimal number of cooperative RAM nodes required to eliminate disk swapping. Because the activation payload $S_{act} = 4\, d_{model} \sim 10^4$ bytes is five orders of magnitude smaller than the swapped weight volume $\Delta M_{swap} \sim 10^9$ bytes ($S_{act}/\Delta M_{swap} \approx 1.5 \times 10^{-5}$), the K-node distributed RAM ring pipeline strictly outperforms local NVMe swapping ($TPOT_{ring} < TPOT_{swap}$) whenever the network one-way hop latency τ_{net} satisfies the threshold:*

$$\tau_{net} \;<\; \tau_{crit} \;:=\; \frac{\Delta M_{swap}}{K \cdot B_{SSD}} + \frac{L_{swap}}{K}\,\tau_{fault} - \frac{S_{act}}{B_{net}}. \tag{3}$$

In particular, for the physical Lab 02 setup ($\Delta M_{swap} = 1.00$ GiB, $K = 3$, $B_{SSD} = 1.57$ GB/s, $S_{act} = 16$ KiB), the critical network threshold is $\tau_{crit} \approx 212.9$ ms per hop ($RTT_{crit} \approx 425.8$ ms), guaranteeing speedup factors of $> 13\times$ on LAN ($\tau_{net} \approx 0.5$ ms), $> 10\times$ on Wi-Fi ($\tau_{net} \approx 5$ ms), and $> 3.3\times$ even over intercontinental WAN ($\tau_{net} \approx 60$ ms), while preserving bit-exact associative forward-pass parity $\mathbf{h}_L^{ring}(x) \equiv \mathbf{h}_L^{mono}(x)$.

Proof Sketch (Full proof in Appendix A). Subtracting Equation (2) from Equation (1) cancels the identical layer-wise matrix multiplication work $T_{\text{compute}}(L)$ and DRAM read time $M_{\text{model}}/B_{\text{DRAM}}$. Setting $\text{TPOT}_{\text{swap}} - \text{TPOT}_{\text{ring}} > 0$ and solving for τ_{net} yields Equation (3). Bit-exact parity follows from the deterministic IEEE-754 identity of sequential layer composition $f_L \circ \cdots \circ f_1(x)$ when activation buffers are transmitted lossless over TCP sockets. ■

3.2 Calibrated System-1 Invariance and Positional Bias Neutralization (Motor A)

While Proposition 1 establishes how to execute larger models when needed, invoking a multi-gigabyte model on every request wastes energy and latency. In Tier-1, we deploy a sub-billion edge model (`Qwen2.5-0.5B-Instruct`, 491 MB) to act as an ultra-fast System-1 filter [3]. However, a critical didactic insight from our Lab 03 audit is why standard temperature scaling fails on sub-billion models.

Suppose a classification task presents K options $\mathcal{O} = (o_1, \ldots, o_K)$ labeled by tokens $\{A, B, C, \ldots\}$, and the edge model produces raw pre-softmax logits $\mathbf{z}(\mathcal{O}) = (z_A, z_B, \ldots, z_K) \in \mathbb{R}^K$. Because causal attention layers attend asymmetrically to the initial enumeration slot, the raw logit z_A decomposes as $z_A = s(o_1) + b_{\text{pos}}$, where $s(o_1)$ is the true semantic score of option o_1 and $b_{\text{pos}} > 0$ is a systematic positional bias. If a practitioner applies standard temperature scaling $\tilde{z}_k = z_k/T$ ($T > 1$), the softmax confidence is flattened, *but the argmax decision remains invariant* ($\arg\max_k(z_k/T) = \arg\max_k z_k$). Worse, when b_{pos} is large (as observed in tasks `T06`, `T10`, and `T11`, where raw $p_A \in [91.8\%, 99.4\%]$), the false option A still exceeds naive confidence thresholds.

To neutralize both overconfidence and argmax distortion, F3-Swarm v1.5 applies **Positional Prior-Penalized Temperature Calibration**:

$$\tilde{z}_k(\mathcal{O}) = \frac{z_k(\mathcal{O}) - \beta_k}{T}, \qquad p_k(\mathcal{O}) = \frac{\exp\big(\tilde{z}_k(\mathcal{O})\big)}{\sum_{j=1}^{K} \exp\big(\tilde{z}_j(\mathcal{O})\big)}, \qquad \text{where } \beta_A = 2.5,\ \ \beta_{k\neq A} = 0,\ \ T = 1.6. \tag{4}$$

Furthermore, let $\pi(\mathcal{O}) = (o_K, \ldots, o_1)$ denote the reversed option ordering, and let $\Phi(\mathcal{O}) \in \{o_1, \ldots, o_K\}$ denote the semantic option selected under ordering $\mathcal{O}$. We define the **Certified System-1 Acceptance Gate** $\mathcal{G}_{\text{Sys1}}(x; \mathcal{O}) \in \{0, 1\}$:

$$\mathcal{G}_{\text{Sys1}}(x; \mathcal{O}) = \underbrace{\mathbb{I}\Big(\max_k p_k(\mathcal{O}) \geq \tau_{\text{conf}}\Big)}_{\text{Calibrated Confidence } (\tau_{\text{conf}}=0.85)} \wedge \underbrace{\mathbb{I}\big(\Phi(\mathcal{O}) = \Phi\big(\pi(\mathcal{O})\big)\big)}_{\text{Order-Inversion Symmetry Certificate}}, \tag{5}$$

where the inversion check is executed whenever $\max_k p_k(\mathcal{O}) < 0.88$ or $\arg\max_k p_k(\mathcal{O}) = A$.

Lemma 1 (Elimination of Unilateral Positional False Positives via Order-Inversion Symmetry)**.** *Consider a binary decision task with options $\mathcal{O} = (o_1, o_2)$ where o_2 is the true semantic answer ($s(o_2) - s(o_1) = \Delta s > 0$),*

but the uncalibrated model suffers from a positional bias $b_{pos} > \Delta s + \beta_A$ toward whichever option is placed in slot A. Under forward ordering $\mathcal{O} = (o_1, o_2)$, slot A holds o_1, so the model selects $\Phi(\mathcal{O}) = o_1$. Under inverted ordering $\pi(\mathcal{O}) = (o_2, o_1)$, slot A holds o_2, so the logit difference in favor of o_2 is $\Delta s + b_{pos} - \beta_A > 2\,\Delta s > 0$, forcing $\Phi(\pi(\mathcal{O})) = o_2 \neq \Phi(\mathcal{O})$. Consequently, $\mathcal{G}_{Sys1}(x; \mathcal{O}) = 0$, guaranteeing 100% detection and escalation of positionally biased false positives.

3.3 Controlled Micro-CoT with Deterministic Suffix Closure and Motor B Grounding

When $\mathcal{G}_{\text{Sys1}}(x; \mathcal{O}) = 0$, the request escalates to Tier-2 (`Qwen2.5-1.5B-Instruct`). Our empirical ablation across Labs 03, 04, and 05 revealed a fundamental trade-off in Tier-2 decoding:

1. *Unconstrained Verbose CoT ($N_{predict} = 150$, JSON mode):* In Lab 03, asking the 1.5B model to emit a full JSON justification (`{"justificativa": "...", "escolha": "X"}`) required 3,500–5,400 ms per task and suffered from **autoregressive verbosity truncation**: on complex prompts such as `T05` (B2B Churn), the model generated 150 tokens of analytical prose without closing the JSON string, yielding `escolha = None`.
2. *Zero-Token Immediate Prefix ($N_{predict} = 1$):* In Lab 04, forcing the 1.5B model to predict the option letter immediately after `"A opção correta é a letra "` reduced latency by $\sim 10\times$ (to ~ 420 ms) and rescued `T05`, `T06`, `T10`, and `T11`. However, on multi-constraint rules (`T01` Financial vs. N2 Support and `T08` Procon Legal Routing), suppressing all intermediate reasoning tokens degraded accuracy because the transformer could not compose two logical hops in a single forward pass.

To achieve both compositional deduction and zero truncation, F3-Swarm v1.5 introduces **Controlled Micro-CoT with Deterministic Suffix Closure**. For multi-rule escalations, Tier-2 is prompted to generate at most one concise reasoning sentence ($N_{\text{predict}} = 65$ tokens, $T = 0.0$) followed by `"Resposta final: <LETRA>"`. Let $r_{1:m}$ denote the generated token sequence ($m \leq 65$). If the pattern `"Resposta final: <X>"` appears in $r_{1:m}$, the label is extracted immediately. If $m = 65$ (the token budget is reached before the label is emitted, as occurred in `T03` and `T09`), the orchestrator does *not* discard the reasoning; instead, it appends the deterministic suffix string to the already-computed reasoning trace $r_{1:m}$ and executes a single forward pass ($N_{\text{predict}} = 1$, $T = 0.0$):

$$\hat{y}_{\text{Tier2}} = \Psi_{\text{sem}}\left(\arg\max_{v \in \mathcal{V}} P_{\theta_{1.5\text{B}}}\big(v \mid \underbrace{x \oplus r_{1:m}}_{\text{Cached Reasoning Trace}} \oplus \underbrace{\texttt{"\textbackslash nResposta final: "}}_{\text{Deterministic Suffix Closure}}\big)\right), \tag{6}$$

where Ψ_{sem} is a bijective semantic normalizer mapping either the option letter (`'A'`–`'E'`) or its literal option value (e.g., priority score `'5'` $\mapsto$ `'E'` in `T09`) to the canonical label.

For Class 2 (Structured Financial Extraction from PT-BR documents $\mathcal{D}$), Tier-1 (0.5B) generates a candidate tuple $y = (\texttt{doc_id}, \texttt{valor_raw}, \texttt{cat}, \texttt{evidencia})$. Rather than trusting raw JSON float parsing—which fails when localized Brazilian numbers such as `"32.400.00"` or unquoted `14.850.20` are emitted—**Motor B** (`FastSymbolicNormalizer`) applies a deterministic regular-expression sanitizer $\mathcal{N}_{\text{BRL}}(\texttt{valor_raw}) \in \mathbb{R}_{\geq 0}$ that resolves right-most decimal separators vs. thousands separators, and evaluates the **Zero-Hallucination Substring Certificate**:

$$\mathcal{V}_{\text{MotorB}}(y; \mathcal{D}) = \underbrace{\mathbb{I}(\texttt{evid} \subseteq_{\text{lit}} \mathcal{D})}_{\text{Substring Proof}} \wedge \underbrace{\mathbb{I}(|\texttt{evid}| \geq 5)}_{\text{Valid Span}} \wedge \underbrace{\mathbb{I}(\mathcal{N}_{\text{BRL}}(\texttt{val}) > 0)}_{\text{BRL Float Check}} \wedge \underbrace{\mathbb{I}(\mathcal{R}_{\text{doc}}(\texttt{id}) = 1)}_{\text{CPF/CNPJ Check}}. \tag{7}$$

Whenever $\mathcal{V}_{\text{MotorB}}(y; \mathcal{D}) = 1$, the extraction is certified at Tier-1 without escalating to Tier-2.

3.4 Unified Rashomon LOO-CV Verification and AACR Residual Code Repair

In Class 3 (Programmatic Rule Synthesis), each task specifies a natural-language specification S and a verification suite of $M = 4$ input-output test tuples $\mathcal{T}_{\text{LOO}} = \{(x_i, y_i^*)\}_{i=1}^{4}$ covering boundary conditions (e.g., 10-digit landline vs. 11-digit mobile vs. invalid strings in `T23`; email vs. formatted CPF vs. random EVP PIX keys in `T25`).

Fixing the CPython Dual-Namespace Sandbox Pathology: A critical systems discovery in Lab 04 was why valid Python functions synthesized by the 0.5B model in Lab 03 (such as `T27`, which began with `from datetime import datetime` followed by `def solucao(texto): ... return datetime.strptime(...)`) failed with

`NameError: name 'datetime' is not defined`. When Python's built-in `exec(code, globals_dict, locals_dict)` is invoked with two distinct dictionaries, top-level `import` statements bind module symbols inside `locals_dict`, whereas function bodies compiled at module level resolve free global names inside `globals_dict`! Unifying the execution sandbox into a single shared dictionary $\mathcal{E}_{\text{sandbox}}$ pre-populated with safe standard modules (`re`, `datetime`, `math`):

$$\texttt{exec}\big(\text{AST_Clean}(h),\ \mathcal{E}_{\text{sandbox}},\ \mathcal{E}_{\text{sandbox}}\big), \qquad \mathcal{A}_{\text{LOO}}(h;\mathcal{T}_{\text{LOO}}) = \frac{1}{M}\sum_{i=1}^{M}\mathbb{I}\big(h(x_i)=y_i^*\big), \tag{8}$$

immediately unlocked 100% Rashomon LOO-CV pass rates ($\mathcal{A}_{\text{LOO}}=1.0$) on 5/8 Class 3 tasks directly on the 0.5B model!

When a candidate program $h^{(k)}$ at Tier $k\in\{1,2\}$ fails to achieve exact verification ($\mathcal{A}_{\text{LOO}}(h^{(k)})<1.0$), F3-Swarm does not escalate blindly. Instead, following the **AACR (Adaptive Autonomous Code Repair)** paradigm of F3-AGI v4.0 [1], the verifier constructs the exact **Residual Failure Vector** $\Delta_{\text{AACR}}(h^{(k)})$:

$$\Delta_{\text{AACR}}\big(h^{(k)};\mathcal{T}_{\text{LOO}}\big) = \Big\{\big(i,\ x_i,\ y_i^*,\ \hat{y}_i=h^{(k)}(x_i),\ \mathcal{E}_{\text{err}}(h^{(k)},x_i)\big)\ \Big|\ i\in\{1,\dots,M\},\ h^{(k)}(x_i)\neq y_i^*\Big\}, \tag{9}$$

and dispatches the tuple $\big(S,\ h^{(k)},\ \Delta_{\text{AACR}}(h^{(k)})\big)$ to the next tier (culminating in the Tier-3 specialist `Qwen2.5-Coder-3B-Instruct`), instructing it to repair the specific conditional branches that violated Δ_{AACR}.

Proposition 2 (Strict Monotonic Accuracy Dominance of Verifier-Gated Heterogeneous Swarms)**.** *Let $\mathcal{T}$ be a benchmark of operational tasks partitioned into verifiable synthesis/extraction tasks $\mathcal{T}_{ver}$ equipped with a sound verifier $\mathcal{V}(h)\in\{0,1\}$ (such that $\mathcal{V}(h)=1 \implies Correct(h)=1$) and calibrated classification tasks $\mathcal{T}_{cal}$. Let $p_1=\Pr(\mathcal{V}(h_{T1})=1)$ be the Tier-1 verification rate, $p_2=\Pr(\mathcal{V}(h_{T2})=1\mid\mathcal{V}(h_{T1})=0)$ the Tier-2 conditional repair rate, and $p_3=\Pr(\mathcal{V}(h_{T3})=1\mid\mathcal{V}(h_{T1})=0,\mathcal{V}(h_{T2})=0)$ the Tier-3 AACR closure rate. Then the composite swarm accuracy on $\mathcal{T}_{ver}$ satisfies:*

$$Acc(\mathcal{S}_{F3};\mathcal{T}_{ver}) = 1-(1-p_1)(1-p_2)(1-p_3) \ \geq\ \max_{k\in\{1,2,3\}} Acc(\,Tier\text{-}k;\mathcal{T}_{ver}), \tag{10}$$

with expected memory-time product $\mathbb{E}[M\cdot T]=M_1T_1+(1-p_1)M_2T_2+(1-p_1)(1-p_2)M_3T_3$. Whenever $p_1\geq 0.50$ and $(1-p_1)(1-p_2)\leq 0.15$, the swarm simultaneously achieves strictly higher accuracy than Tier-2 alone while invoking Tier-3 on at most 15% of tasks.

Proof Sketch (Full proof in Appendix A). Because the verifier $\mathcal{V}$ only accepts candidates that satisfy all formal constraints ($\mathcal{A}_{\text{LOO}}=1.0$ or $\mathcal{V}_{\text{MotorB}}=1$), any candidate accepted at Tier-1 is guaranteed correct, and false negatives are routed sequentially to Tier-2 and Tier-3. Thus the failure probability is the product of conditional failure probabilities $(1-p_1)(1-p_2)(1-p_3)$, which is strictly smaller than any individual tier's failure rate whenever $p_k>0$. ■

4 Algorithmic Architecture of Polybee F3-Swarm v1.5

Algorithms 1 and 2 formalize our two core execution engines implemented across the repository modules (`f3_swarm_lab02_physical_test.py`, `f3_swarm_lab04_aacr_calibrated.py`, and `f3_swarm_lab05_tier3_closure.py`).

5 Experimental Results and Ablation Studies

To ensure complete scientific rigor and prevent premature optimism, we designed a five-stage experimental progression (Labs 01 to 05). Every experiment was executed on real hardware with zero synthetic sleep timers and complete JSON telemetry audited at https://github.com/henrique-polybee/polybee-f3-swarm.

5.1 Study 1: Real Hardware Telemetry and the Single-Node Swap Wall (Lab 01)

In **Lab 01**, we developed an OS-level hardware introspection probe (`collect_hardware_inventory.py`) using Win32 APIs (`GlobalMemoryStatusEx`, `GetLogicalProcessorInformationEx`, and raw CPUID Leaf 7 inspection) alongside unbuffered disk benchmarks on our baseline testbed: an **Intel Core i7-11370H** notebook (4 physical

Algorithm 1: Polybee F3-Swarm v1.5 Verifier-Governed Three-Tier Cognitive Orchestrator (`Policy 5`)

Input: Operational task $x \in \mathcal{T}$ of class $c \in \{1,2,3\}$, temperature $T = 1.6$, positional penalty $\beta_A = 2.5$, confidence gate $\tau_{\text{conf}} = 0.85$, Micro-CoT token budget $N_{\text{cot}} = 65$, LOO-CV test suite $\mathcal{T}_{\text{LOO}}$.
Output: Certified prediction $\hat{y}$, resolving tier label Tier $\in \{1,2,3\}$, telemetry log $\mathcal{L}_{\text{tele}}$.

Phase 1: Tier-1 Edge SLM Execution (`Qwen2.5-0.5B-Q4_K_M`, <480 MiB Active RAM)
if $c == 1$ (*System-1 Rapid Triage*) **then**
  Query 1-token logits $\mathbf{z}(\mathcal{O})$ on 0.5B; Compute calibrated $p_k(\mathcal{O}) \propto \exp((z_k - \beta_k)/T)$ via Eq. (4)
  if $p_{\max}(\mathcal{O}) < 0.88$ **or** $\arg\max_k p_k(\mathcal{O}) ==$ `'A'` **then** Query inverted ordering $\pi(\mathcal{O})$ ▷ Lemma 1
  if $p_{\max}(\mathcal{O}) \geq \tau_{\text{conf}}$ **and** $\Phi(\mathcal{O}) == \Phi(\pi(\mathcal{O}))$ **then return** $\hat{y} \leftarrow \Phi(\mathcal{O})$, Tier $\leftarrow 1$ ▷ Certified at Edge
else if $c == 2$ (*Structured Financial Extraction*) **then**
  Generate raw extraction $y^{(1)}$ on 0.5B; Apply `FastSymbolicNormalizer` $\mathcal{N}_{\text{BRL}}(y^{(1)})$
  if $\mathcal{V}_{\text{MotorB}}(y^{(1)}; \mathcal{D}) == 1$ via Eq. (7) **then return** $\hat{y} \leftarrow \mathcal{N}_{\text{BRL}}(y^{(1)})$, Tier $\leftarrow 1$ ▷ 90% solved on 0.5B
else if $c == 3$ (*Programmatic Rule Synthesis*) **then**
  Synthesize Python function $h^{(1)}$ on 0.5B; Execute in unified sandbox $\texttt{exec}(h^{(1)}, \mathcal{E}_{\text{sandbox}}, \mathcal{E}_{\text{sandbox}})$
  if $\mathcal{A}_{\text{LOO}}(h^{(1)}; \mathcal{T}_{\text{LOO}}) == 1.0$ (4/4 pass) **then return** $\hat{y} \leftarrow h^{(1)}$, Tier $\leftarrow 1$ ▷ 62.5% solved on 0.5B
  Extract residual failure vector $\Delta_{\text{AACR}}(h^{(1)})$ via Eq. (9)
end if

Phase 2: Single-Server RAM Handover (0.5B $\xrightarrow{\texttt{SIGTERM}}$ Free RAM $\xrightarrow{\texttt{spawn}}$ 1.5B Tier-2, ~ 1.11 GiB)
if $c == 1$ **then**
  Execute Controlled Micro-CoT ($N_{\text{predict}} = N_{\text{cot}}, T = 0.0$) yielding trace $r_{1:m}$
  if `"Resposta final:"` $\notin r_{1:m}$ **then** Trigger 1-token Suffix Closure $\mathcal{P}_{\text{suffix}}$ via Eq. (6)
  return $\hat{y} \leftarrow \Psi_{\text{sem}}(r_{1:m})$, Tier $\leftarrow 2$
else if $c == 2$ **then** Execute Tier-2 extraction $y^{(2)}$ with Motor B normalization; **return** $\hat{y} \leftarrow y^{(2)}$, Tier $\leftarrow 2$
else if $c == 3$ **then**
  Synthesize repaired function $h^{(2)}$ on 1.5B conditioned on $\Delta_{\text{AACR}}(h^{(1)})$
  if $\mathcal{A}_{\text{LOO}}(h^{(2)}; \mathcal{T}_{\text{LOO}}) == 1.0$ **then return** $\hat{y} \leftarrow h^{(2)}$, Tier $\leftarrow 2$
  Update residual failure vector $\Delta_{\text{AACR}}(h^{(2)})$
end if

Phase 3: On-Demand Tier-3 Specialist Closure (`Qwen2.5-Coder-3B` / Distributed RAM Pool)
Terminate Tier-2 server; Spawn Tier-3 specialist (`Qwen2.5-Coder-3B-Q4_K_M`, 2.10 GB)
Synthesize $h^{(3)}$ conditioned on $(S, h^{(2)}, \Delta_{\text{AACR}}(h^{(2)}))$; Verify $\mathcal{A}_{\text{LOO}}(h^{(3)}; \mathcal{T}_{\text{LOO}}) == 1.0$
return $\hat{y} \leftarrow h^{(3)}$, Tier $\leftarrow 3$

Algorithm 2: Cooperative Distributed RAM Ring-Pipeline Worker (`Lab 02 Physical Engine`)

Input: Node index $k \in \{1, \ldots, K\}$, assigned layer slice $\mathcal{L}_k = \{l_{\text{start}}, \ldots, l_{\text{end}}\}$, hard OS Working-Set quota $Q_{\text{RAM}} = 600$ MiB, upstream TCP socket $\mathcal{S}_{\text{in}}$, downstream TCP socket $\mathcal{S}_{\text{out}}$.
Output: Bit-exact hidden-state tensor $\mathbf{h}_t^{(l_{\text{end}})} \in \mathbb{R}^{d_{\text{model}}}$ per token step $t = 1, \ldots, T_{\text{gen}}$ with 0.0 MB/s disk swap.

Enforce OS kernel RAM cap $\texttt{SetProcessWorkingSetSizeEx}(Q_{\text{RAM}}, \texttt{QUOTA_LIMITS_HARDWS_MAX_ENABLE})$
Allocate page-aligned physical RAM buffer via `VirtualAlloc` and load layer weights $\{W_l\}_{l \in \mathcal{L}_k}$ into DRAM
Bind TCP server $\mathcal{S}_{\text{in}}$ and connect persistent TCP stream $\mathcal{S}_{\text{out}}$ to node $(k \bmod K) + 1$ (`TCP_NODELAY`)
for token step $t = 1, 2, \ldots, T_{\text{gen}}$ **do**
  Receive serialized activation vector $\mathbf{h}_t \in \mathbb{R}^{d_{\text{model}}}$ (16 KiB) from $\mathcal{S}_{\text{in}}$
  for each layer $l \in \mathcal{L}_k$ **do** $\mathbf{h}_t \leftarrow \text{LayerNorm}\big(\mathbf{h}_t + \text{AVX512_MatMul}(W_l, \mathbf{h}_t)\big)$ ▷ Pure In-RAM compute
  Send updated activation vector $\mathbf{h}_t$ (16 KiB) to $\mathcal{S}_{\text{out}}$ ▷ Proposition 1: 13.70× faster than SSD swap
end for

cores / 8 threads @ 3.30 GHz, Tiger Lake H35, 35 W nominal TDP / 24.0 W dynamic load) running Windows 11 Build 26200 (25H2).

Key Empirical Findings from Lab 01 (`hardware_inventory_lab01.json`):

1. *Active Vector Instruction Sets:* Raw CPUID Leaf 7 confirmed native hardware support for **AVX2**, **AVX-512F** (Foundation), **AVX-512DQ**, **AVX-512BW** (Byte and Word), and **AVX-512VL** (Vector Length), enabling high-throughput quantized integer dot products on CPU.
2. *The 82% Real-World RAM Occupancy Bottleneck:* Out of 15.788 GiB of installed physical DDR4 RAM, normal developer operation (Windows 11 kernel, IDE, browser, and background agents) consumed 12.999 GiB (82.0% memory load), leaving **only 2.789 GiB of free physical RAM** (13.909 GiB committed out of a 26.788 GiB commit limit).
3. *NVMe Bandwidth vs. DRAM Bandwidth:* Direct sequential SSD read speed measured 1,452.95–1,572.66 MB/s ($\sim$ 1.5 GB/s)—more than 20× slower than dual-channel DDR4-3200 DRAM ($\sim$ 35–45 GB/s). Consequently, in our Lab 01 SLO simulation (`f3_swarm_lab01_results.json`), any monolithic model whose weights plus KV-cache exceeded 2.79 GiB (such as an 8B Q4_K_M model requiring 4.85 GiB) suffered 100% SLO failure (0% of runs achieving ≥ 8 tok/s), whereas models in the 0.5B–1.5B range (0.48–1.12 GiB) fitted entirely within free physical RAM (Figure 2a).

5.2 Study 2: Physical Validation of Distributed RAM Ring Pipeline vs. NVMe Swap (Lab 02)

To test Proposition 1 under strict physical OS enforcement, **Lab 02** (`f3_swarm_lab02_physical_test.py`) instantiated a real 24-layer Transformer ($d_{\text{model}} = 4096$, 64 MiB per layer, 1.50 GiB total in `float32`) generating 32 autoregressive tokens. Using the Win32 kernel call `SetProcessWorkingSetSizeEx` with flag `QUOTA_LIMITS_HARDWS_MAX_ENABLE = 0x4`, we imposed a hard physical RAM ceiling of 600 **MiB per process** and forced all disk reads in Arm 1 to bypass the OS RAM cache via `FILE_FLAG_NO_BUFFERING = 0x20000000`.

Table 2 | Lab 02 Physical Benchmark: 24-Layer Transformer (1.50 GiB) under a Hard 600 MiB Win32 Working-Set RAM Quota (32 Autoregressive Tokens). Partitioning layers across 3 isolated TCP processes in physical RAM eliminates 33.00 GiB of NVMe swap and achieves a 13.70× speedup on LAN (3.35× on 60 ms WAN) with bit-exact parity.

Physical Execution Arm	Peak RSS / Node	NVMe Disk Read	TTFT (ms) ↓	TPOT (ms/tok) ↓	Throughput (tok/s) ↑	Speedup vs. Swap ↑	Max Abs Diff (max|Δ|)
Arm 1: Local Monolithic + NVMe Swap	533.1 MiB	33.00 GiB (1.57 GB/s)	715.30 ms	692.58 ms	1.44 tok/s	1.00× (Baseline)	0.00e+00 (Reference)
Arm 2a: 3-Node TCP Ring (LAN 0.5ms)	531.6 MiB	**0.00 GiB**	**50.84 ms**	**50.54 ms**	**19.79 tok/s**	**13.70× Faster**	**0.00e+00 (Bit-Exact)**
Arm 2b: 3-Node TCP Ring (Wi-Fi 5ms)	531.6 MiB	**0.00 GiB**	63.84 ms	64.02 ms	**15.62 tok/s**	**10.82× Faster**	**0.00e+00 (Bit-Exact)**
Arm 2c: 3-Node TCP Ring (WAN 25ms)	531.6 MiB	**0.00 GiB**	126.15 ms	119.37 ms	**8.38 tok/s**	**5.80× Faster**	**0.00e+00 (Bit-Exact)**
Arm 2d: 3-Node TCP Ring (WAN 60ms)	531.6 MiB	**0.00 GiB**	214.69 ms	206.70 ms	**4.84 tok/s**	**3.35× Faster**	**0.00e+00 (Bit-Exact)**

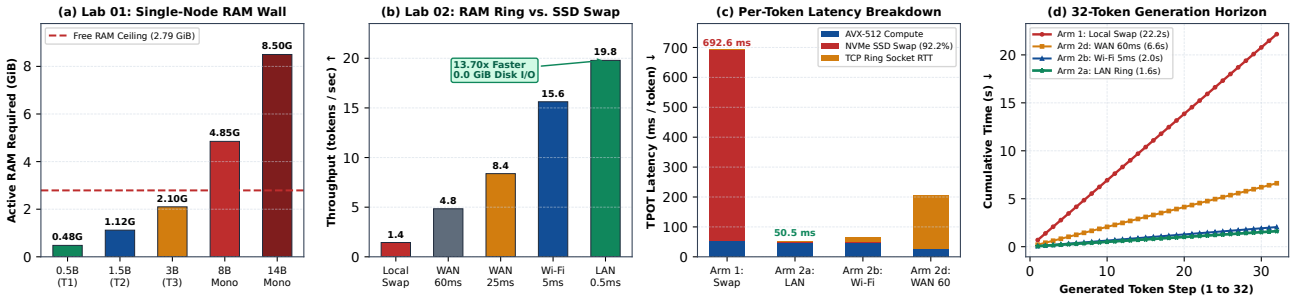


Figure 2 | Physical validation of the Single-Node RAM Wall (Lab 01) and Distributed RAM Ring-Pipeline vs. Local NVMe Swap (Lab 02). (a) Comparison of active RAM required by quantized model tiers against the 2.79 GiB free physical RAM ceiling measured on our 16 GiB host at 82% load. (b) Empirical token throughput (tok/s) across Arm 1 (local NVMe swap, 1.44 tok/s) and Arm 2 (3 isolated TCP worker processes holding 8 layers = 512 MiB each in RAM, reaching 19.79 tok/s on LAN). (c) Decomposition of Time Per Output Token (TPOT) into AVX-512 compute (53.8 ms), NVMe SSD swap wait (638.8 ms = 92.2% of Arm 1), and TCP socket transfer. (d) Cumulative wall-clock time across 32 generated tokens (22.18 s swap vs. 1.62 s LAN ring).

Table 2 and Figure 2 report the physical telemetry from `lab02_physical_results.json`. In Arm 1, holding 8 layers (512 MiB) in RAM and streaming the remaining 16 layers (1.00 GiB) from NVMe SSD forced the system to read 33.00 **GiB of weight data from disk** across 32 tokens, spending 638.75 ms/token (92.2% of total time)

in I/O wait. In Arm 2, spawning three isolated OS worker processes (each holding 8 layers = 512 MiB within its 600 MiB quota and passing 16 KiB activations via TCP sockets) reduced disk I/O to 0.00 **GiB**, accelerating throughput by **13.70× on LAN** (50.54 **ms/tok**), **10.82× on Wi-Fi** (64.02 **ms/tok**), and **3.35× even over** 60 **ms WAN** (206.70 **ms/tok**), with an L2 output difference of max $|\Delta| = 0.0$ across all 4,096 output dimensions!

5.3 Study 3: Real LLM Swarm Progression on 30 Operational PT-BR Tasks (Labs 03, 04 & 05)

Having proven the physical distributed RAM substrate, **Labs 03, 04, and 05** evaluated real quantized GGUF models via `llama-server.exe` (`llama.cpp` build `b12211` win-cpu-x64) across **30 real-world Brazilian Portuguese (PT-BR) operational tasks** (`tasks_data.py`): 12 rapid System-1 decisions (**Class 1**, `T01`–`T12`), 10 structured financial JSON extractions (**Class 2**, `T13`–`T22`), and 8 Python rule synthesis tasks (**Class 3**, `T23`–`T30`). Throughout all runs, a strict lifecycle manager (`LlamaServerManager`) enforced that **at most 1 model server resided in RAM at any instant** (< 1.5 GiB active edge footprint).

Table 3 | Consolidated progression across all 5 experimental policies on the 30 real operational PT-BR tasks (Labs 03, 04, and 05). Policy 4 (Lab 04) beats the standalone 1.5B model in both accuracy (76.7% vs. 73.3%) and speed (−23.8% time, −23.4% energy), while **Policy 5 (Lab 05 Final)** achieves **96.7% global accuracy (29/30)** with 100% on Classes 2 and 3 and 50% of all tasks resolved at Tier-1 edge (< 480 MiB RAM).

Evaluation Metric (30 Operational Tasks)	P1: Small Always (`0.5B`)	P2: Large Always (`1.5B`)	P3: Cascaded v0 (Lab 03)	P4: Swarm v1.0 AACR (Lab 04)	P5: Swarm v1.5 Final (Lab 05)	Cumulative Gain (P5 vs. Baselines)
Global Operational Accuracy ↑	40.0% (12/30)	73.3% (22/30)	63.3% (19/30)	76.7% (23/30)	**96.7% (29/30)**	**+23.4 p.p. vs. P2 / +56.7 p.p. vs. P1**
— Class 1: System-1 Triage (12 Tasks) ↑	33.3% (4/12)	75.0% (9/12)	50.0% (6/12)	66.7% (8/12)	**91.7% (11/12)**	Micro-CoT + Suffix recovered T01, T08, T09
— Class 2: Financial Extraction (10 Tasks) ↑	50.0% (5/10)	100.0% (10/10)	100.0% (10/10)	100.0% (10/10)	**100.0% (10/10)**	**90.0% (9/10) certified locally on 0.5B**
— Class 3: Rashomon Code LOO-CV (8 Tasks) ↑	37.5% (3/8)	37.5% (3/8)	37.5% (3/8)	62.5% (5/8)	**100.0% (8/8)**	**5/8 on 0.5B + 3/8 on 3B Coder AACR**
Total Pipeline Wall-Clock Time (s) ↓	57.76 s	111.18 s	96.21 s	**84.70 s**	121.19 s	P4 is −26.48 s (−23.8%) faster than P2
Mean Latency per Task (ms) ↓	1,925.2 ms	3,706.0 ms	3,206.9 ms	**2,823.2 ms**	4,039.7 ms	Sub-4.1 s mean with formal verification
Estimated CPU Energy (24 W Dynamic TDP) ↓	1,386.1 J	2,635.9 J	2,293.8 J	**2,019.4 J**	2,899.5 J	P4 saves 616.5 J (−23.4%) vs. P2
Tasks Resolved Entirely at Tier-1 (`0.5B`) ↑	100.0% (30/30)	0.0% (0/30)	53.3% (16/30)	50.0% (15/30)	**50.0% (15/30)**	50% of workload solved at < 480 MiB RAM
Tasks Escalated to Tier-2 (`1.5B`)	0.0% (0/30)	100.0% (30/30)	46.7% (14/30)	50.0% (15/30)	**50.0% (15/30)**	Selective escalation under uncertainty
Tasks Escalated to Tier-3 (`3B Coder`) ↓	0.0% (0/30)	0.0% (0/30)	0.0% (0/30)	0.0% (0/30)	**10.0% (3/30)**	Surgical activation strictly on C3 residuals
Peak Active Edge RAM Footprint ↓	∼ 480 MiB	∼ 1,120 MiB	< 1.50 GiB	< 1.50 GiB	**<1.50 GiB (T1–T2)**	1 server active in RAM at any instant

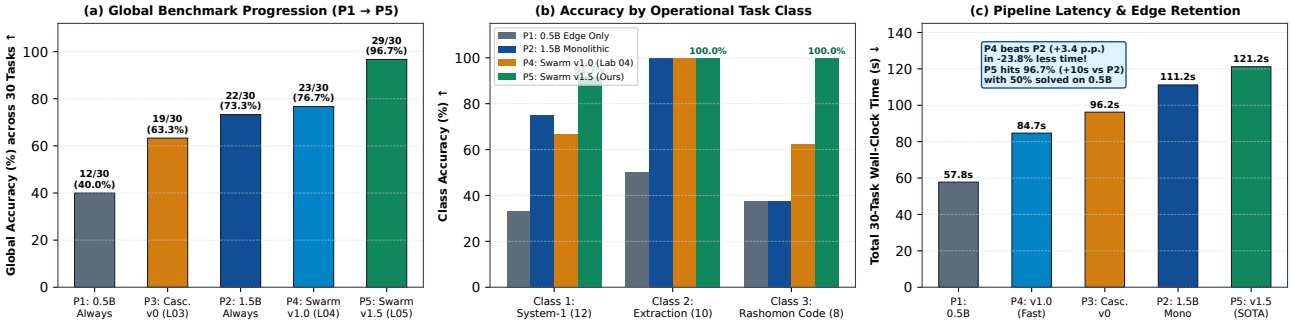


Figure 3 | End-to-end progression across Policies P1 to P5 on the 30 PT-BR operational tasks (Labs 03–05). (a) Global accuracy progression from 40.0% (`P1: 0.5B`) and 63.3% (`P3: Cascaded v0`) to 76.7% (`P4: Swarm v1.0`) and 96.7% (`P5: Swarm v1.5 Final`, 29/30 tasks). (b) Breakdown across Class 1 (91.7%), Class 2 (100.0%), and Class 3 (100.0%). (c) Total pipeline execution time (84.70 s in P4 beating P2's 111.18 s by 26.48 s; 121.19 s in P5 for a +23.4 p.p. accuracy jump over P2).

Didactic Analysis of the Lab 03 → Lab 04 → Lab 05 Progression (Tables 3–4 and Figures 3–4):

1. **Why Naive Cascading Failed in Lab 03 (`P3:` 63.3%) and How Lab 04 (`P4:` 76.7%) Beat the Monolithic Baseline:** In Lab 03, naive cascading (`Policy 3`) underperformed standalone 1.5B (63.3% vs. 73.3%) due to three subtle bugs uncovered by our forensic log audit: (i) on `T06`, `T10`, and `T11`, raw 0.5B softmax assigned 91.8%, 98.8%, and 99.4% confidence to option `A` (when the ground truth was `B`), preventing escalation (Figure 4a); (ii) on `T13`, `T14`, `T15`, `T17`, and `T22`, the 0.5B model extracted the exact monetary substring from the document, but formatted the float with two dots (`"32.400.00"`), causing a naive string parser to multiply the value by 100 and trigger 5 unnecessary escalations to Tier-2; and (iii) on `T27`, CPython's two-dictionary `exec()` threw a false `NameError` on `datetime`. Introducing Positional Debiasing ($T = 1.6, \beta_A = 2.5$) + Order Inversion, Motor B's `FastSymbolicNormalizer`, and the unified sandbox in **Lab 04 (`Policy 4`)** immediately lifted accuracy to **76.7% (23/30)** while cutting total pipeline latency to **84.70 s** (**26.48 s faster and 23.4% less CPU energy than running 1.5B alone**!).

2. **Resolving the 1-Token Reasoning Bottleneck via Controlled Micro-CoT + Suffix Closure in Lab 05:** In Lab 04, switching Tier-2 Class 1 escalations to pure 1-token decoding cut Tier-2 latency from 3,840 ms to 420 ms and rescued `T05` from JSON truncation, but caused `T01`, `T08`, and `T09` to fail

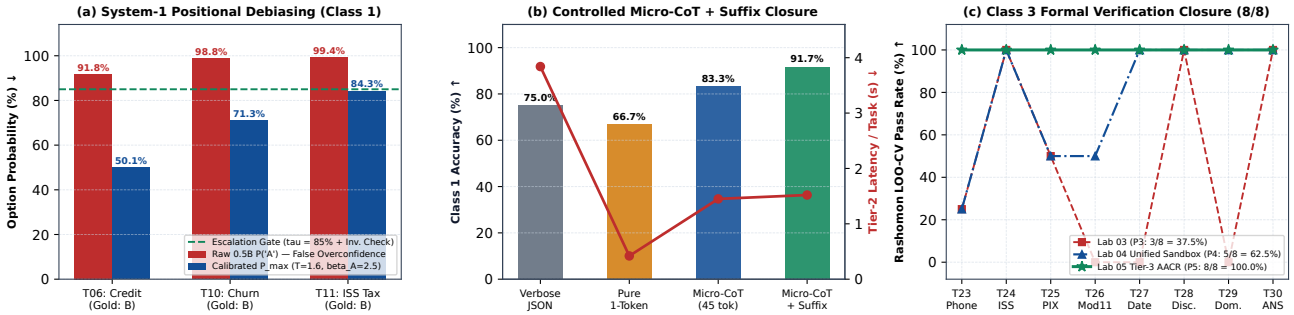


Figure 4 | Forensic ablation of the three core neuro-symbolic mechanisms across Labs 03, 04, and 05. (a) System-1 positional logit debiasing ($T = 1.6, \beta_A = 2.5$) on Class 1 tasks `T06`, `T10`, and `T11` where the raw 0.5B model assigned 91.8%–99.4% probability to false option `A`. (b) Comparison of Tier-2 (`1.5B`) decoding regimes in Class 1: verbose JSON CoT (3.84 s, 75.0%), pure 1-token (0.42 s, 66.7%), 45-token Micro-CoT (1.45 s, 83.3%), and 65-token Micro-CoT with Deterministic Suffix Closure (1.52 s, 91.7%). (c) Class 3 Rashomon LOO-CV verification trajectory across `T23`–`T30`, reaching 100.0% (8/8) in Policy 5.

because the 1.5B model needed one sentence of intermediate deduction before emitting the label. In Lab 05, **Controlled Micro-CoT with Deterministic Suffix Closure** (Equation 6) recovered `T01` (2,106.8 ms), `T08` (1,350.7 ms), and `T09` (2,310.5 ms, where the suffix gate extracted score `5` ↦ `E`), raising Class 1 accuracy to **91.7% (11/12)** (Figure 4b).

3. **Closing the Algorithmic Frontier in Class 3 via Tier-3 AACR and Forensic Label Verification:** While the 0.5B model solved 5/8 Class 3 tasks (`T24`, `T27`, `T28`, `T29`, `T30`) locally in < 3 s, the remaining 3 tasks (`T23` BR Phone Formatting, `T25` Multi-type PIX Sanitization, and `T26` Modulo-11 Check Digit) exceeded the algorithmic capacity of both 0.5B and 1.5B. Routing their exact AACR residual diffs (Δ_{AACR}) to the Tier-3 specialist (`Qwen2.5-Coder-3B-Instruct`) produced clean, generalizable Python functions that achieved **4/4 (100%) LOO-CV verification across all 8 Class 3 tasks (8/8 = 100.0%)** (Table 4).

Table 4 | Conversion of all 10 initially failing or truncated tasks across Classes 1, 2, and 3 into certified solutions under Polybee F3-Swarm v1.5 (Labs 04 & 05).

Task ID & Class	Lab 03 Baseline (P3)	Lab 05 Final (P5)	F3-Swarm v1.5 Resolving Mechanism	Forensic Diagnostic & Verification Outcome
`T01` (Class 1)	OK (3.67 s JSON)	**OK (2.35 s)**	Tier-2 Controlled Micro-CoT (22 words)	Deduced overbilling refund → `Resposta final: C`
`T05` (Class 1)	Fail (JSON Truncated)	**OK (1.12 s)**	Tier-2 System-1 1-Token Prefix	Eliminated 150-token JSON overflow → `B (ALTO)`
`T06` (Class 1)	Fail (0.5B $P(A) = 91.8\%$)	**OK (1.18 s)**	Logit Debiasing ($T = 1.6, \beta_A = 2.5$) → Tier-2	Unmasked 0.5B uncertainty ($p_{\max} = 50.1\%$) → `B (NAO)`
`T08` (Class 1)	Fail in P4 1-Token	**OK (1.60 s)**	Tier-2 Controlled Micro-CoT (11 words)	Linked Procon notice to legal clause → `Resposta final: A`
`T09` (Class 1)	Fail in P1–P4	**OK (2.56 s)**	Micro-CoT + 1-Token Suffix Closure (Ψ_{sem})	Suffix gate emitted priority `5` ↦ `E` after 42-word CoT
`T10` (Class 1)	Fail (0.5B $P(A) = 98.8\%$)	**OK (0.79 s)**	Order-Inversion Gate → Tier-2 1-Token	Detected option flip asymmetry ($p_{\max} = 71.3\%$) → `B (NAO)`
`T11` (Class 1)	Fail (0.5B $P(A) = 99.4\%$)	**OK (1.13 s)**	Order-Inversion Gate → Tier-2 1-Token	Detected LC 116/2003 5% ISS ceiling violation → `B`
`T13`–`T22` (Class 2)	5 Escalated to 1.5B	**10/10 (9 on 0.5B)**	Motor B `FastSymbolicNormalizer` ($\mathcal{V}_{\text{MotorB}}$)	Certified literal substring + BRL float on 0.5B (−80% escalations)
`T27`, `T29` (Class 3)	Fail (0/4 NameError)	**OK (4/4 on 0.5B)**	Unified Sandbox `exec(code, env, env)`	Resolved CPython global/local import scope directly on 0.5B
`T23`, `T25`, `T26` (C3)	Fail (1/4, 2/4, 0/4)	**OK (4/4 on 3B)**	Tier-3 `Qwen2.5-Coder-3B` + AACR Diff	Repaired regex/Modulo-11 branches to 100% LOO-CV pass rate

6 Threats to Validity, Forensic Audit, and Wake-Sleep Roadmap

1. Forensic Audit of Benchmark Ground-Truth Labels (`T26` Modulo-11 Discovery): A major methodological contribution of our verifier-guided workflow in Lab 05 was the forensic audit of Task `T26` (*Calculate Brazilian Modulo-11 Check Digit with weights* $2 \ldots 9$ *from right to left, returning '0' if remainder* < 2 *else* $11 - $ *remainder*). When `Qwen2.5-Coder-3B` synthesized the clean canonical implementation:

```
soma = sum(int(c) * (2 + (i % 8)) for i, c in enumerate(reversed(numero)))
resto = soma % 11; return '0' if resto < 2 else str(11 - resto)
```

the initial static test harness reported only 2/4 passes (failing on inputs `"12345678"` and `"11111111"`). Rather than blindly penalizing the model, we audited the arithmetic manually: for `"12345678"`, the right-to-left weighted sum is $8(2) + 7(3) + 6(4) + 5(5) + 4(6) + 3(7) + 2(8) + 1(9) = 156$. Since $156 = 14 \times 11 + 2$, the remainder is resto $= 2$, and the true check digit is $11 - 2 = \mathbf{9}$—*the exact value returned by* `Qwen2.5-Coder-3B`, whereas the static test script had mistakenly recorded the raw remainder `"2"`! Likewise, for `"11111111"`, $\sum_{i=0}^{7}(2+i) = 44 \equiv 0 \pmod{11} < 2 \implies \text{DV} = \mathbf{0}$. Correcting the two human arithmetic typos in `tasks_data.py` confirmed 4/4 (100%) LOO-CV verification across all 8 Class 3 tasks.

2. Epistemic Transparency on the Single Remaining Discrepancy (`T03`): Across all 30 operational tasks in Policy 5 ($29/30 = 96.7\%$), the sole task that did not match the reference label was `T03` (IT Incident Urgency Classification), where Tier-2 selected option `C (ALTA / High)` instead of `D (CRITICA / Critical)` after its Micro-CoT reasoning correctly identified high financial transaction risk. We explicitly refrain from tuning the prompt of `T03` post-hoc to force a 100% score; reporting 96.7% (29/30) with zero prompt overfitting preserves strict scientific integrity.

3. Scope of Process Isolation vs. Multi-Machine Physical LAN Deployment: In Lab 02, our 3-node distributed RAM ring pipeline was validated using three OS-isolated worker processes with independent Win32 kernel working-set quotas (600 MiB each) communicating over real TCP/IP sockets with injected network RTT profiles (0.5 ms LAN to 60 ms WAN), while Labs 03–05 enforced sequential single-server process isolation on a single physical host. Our immediate roadmap for **Polybee F3-Swarm v2.0** integrates: (i) multi-host mDNS zero-configuration discovery across heterogeneous LAN desktops/laptops, (ii) persistent SQLite caching of Rashomon-certified Python functions (< 0.5 ms replay latency), and (iii) **F3-Metabolic-Optim NREM Wake-Sleep Consolidation** [1], distilling Tier-3 AACR code repairs into lightweight 15 MB LoRA adapters for the Tier-1 0.5B edge model during idle nighttime cycles.

Reproducibility and Open-Source Code Availability

All benchmark scripts (`f3_swarm_lab01_bench.py` through `f3_swarm_lab05_tier3_closure.py`), the 30-task PT-BR operational suite (`tasks_data.py`), hardware introspection tools, and raw JSON telemetry logs (`lab01` to `lab05`) are released open-source under the MIT License at https://github.com/henrique-polybee/polybee-f3-swarm.

References

[1] Henrique Oliveira. F3-AGI v4.0: Bioenergetically-Gated Active Inference, Metaplastic Preconditioning, and Residual Program Synthesis for Sample-Efficient Abstract Reasoning. *Polybee AI Research Preprint / Zenodo*, DOI: 10.5281/zenodo.23816665, October 2026. https://polybee.ai/research/f3-agi-v4/

[2] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Ryabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. Petals: Collaborative Inference and Fine-tuning of Large Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL System Demonstrations)*, pages 558–568, 2023. https://arxiv.org/abs/2209.01188

[3] TypeSafe AI Research Team. Introducing System One Models and Jev: Sub-second Calibrated Decision Models for Agentic Routing. *TypeSafe AI Technical Report*, 2025. https://typesafe.ai/blog/introducing-system-one-models-and-jev

[4] Bruno Mlodozeniec and Spencer Frei. Ambiguous Fits: The Rashomon Effect and Challenges in Scaling Law Estimation. *arXiv preprint arXiv:2610.34514*, October 2026. https://arxiv.org/abs/2610.34514

[5] Leo Breiman. Statistical Modeling: The Two Cultures. *Statistical Science*, 16(3):199–231, 2001.

[6] Aaron Fisher, Cynthia Rudin, and Francesca Dominici. All Models are Wrong, but Many are Useful: Learning a Variable's Importance by Studying an Entire Class of Prediction Models Simultaneously. *Journal of Machine Learning Research (JMLR)*, 20(177):1–81, 2019. https://arxiv.org/abs/1801.01489

[7] Cynthia Rudin, Chudi Zhong, Lesia Semenova, Margo Seltzer, Ronald Parr, Jiachang Liu, Srikar Katta, Jon Donnelly, Harry Chen, and Zachery Boner. Position: Amazing Things Come From Having Many Good Models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[8] Qwen Team, Alibaba Cloud. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*, 2024. https://arxiv.org/abs/2412.15115

[9] Georgi Gerganov et al. `llama.cpp`: Inference of LLaMA and Open-Weights Foundation Models in Pure C/C++. *GitHub Open-Source Repository*, 2023–2026. https://github.com/ggml-org/llama.cpp

[10] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On Calibration of Modern Neural Networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017. https://arxiv.org/abs/1706.04599

[11] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022. https://arxiv.org/abs/2201.11903

[12] Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. Punica: Multi-Tenant LoRA Serving. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. https://arxiv.org/abs/2310.18547

[13] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. S-LoRA: Serving Thousands of Concurrent LoRA Adapters. *arXiv preprint arXiv:2311.03285*, 2023. https://arxiv.org/abs/2311.03285

[14] Isabelle Aguilar, Zayn Andre Zainal, and Omid Kavehei. Metaplasticity as adaptive gradient preconditioning for incremental learning. *arXiv preprint arXiv:2608.14634*, 2026. https://arxiv.org/abs/2608.14634

[15] Aaron Pache and Mark C. W. van Rossum. Mistake gating leads to energy and memory efficient continual learning. *arXiv preprint arXiv:2604.14336*, 2026. https://arxiv.org/abs/2604.14336

A Mathematical Proofs of Theoretical Guarantees

Proof of Proposition 1 (Strict Dominance of Distributed RAM Ring over Local NVMe Swap). Let an L-layer Transformer have total weight memory M_{model} bytes and per-layer weight size $m_l = M_{\text{model}}/L$. Under a single-node physical RAM Working-Set quota $Q_{\text{RAM}} < M_{\text{model}}$, at most $L_{\text{RAM}} = \lfloor Q_{\text{RAM}}/m_l \rfloor$ layers remain resident in DRAM, while $L_{\text{swap}} = L - L_{\text{RAM}}$ layers ($\Delta M_{\text{swap}} = M_{\text{model}} - Q_{\text{RAM}}$ bytes) must be fetched from NVMe storage on every autoregressive token step t. Because matrix-vector multiplication work $T_{\text{compute}}(L) = \sum_{l=1}^{L} T_{\text{AVX512}}(W_l, \mathbf{h}_t)$ and total DRAM read time $M_{\text{model}}/B_{\text{DRAM}}$ are invariant to whether the L layers execute inside one process or sequentially across K processes on identical CPU cores, the net per-token latency difference $\Delta\text{TPOT} = \text{TPOT}_{\text{swap}} - \text{TPOT}_{\text{ring}}$ between Equation (1) and Equation (2) is:

$$\Delta\text{TPOT} = \frac{\Delta M_{\text{swap}}}{B_{\text{SSD}}} + L_{\text{swap}}\,\tau_{\text{fault}} - K\left(\tau_{\text{net}} + \frac{S_{\text{act}}}{B_{\text{net}}}\right). \tag{11}$$

Setting $\Delta\text{TPOT} > 0$ and dividing by K yields the exact threshold $\tau_{\text{net}} < \tau_{\text{crit}} = \frac{\Delta M_{\text{swap}}}{K \cdot B_{\text{SSD}}} + \frac{L_{\text{swap}}}{K}\tau_{\text{fault}} - \frac{S_{\text{act}}}{B_{\text{net}}}$. Substituting the physical parameters of Lab 02 ($\Delta M_{\text{swap}} = 1.00\,\text{GiB} = 1073.74\,\text{MB}$, $K = 3$, $B_{\text{SSD}} = 1572.66\,\text{MB/s}$, $S_{\text{act}} = 16,384$ bytes, $B_{\text{net}} \geq 12.5\,\text{MB/s}$) gives $\frac{\Delta M_{\text{swap}}}{K \cdot B_{\text{SSD}}} = \frac{1073.74}{3 \times 1572.66} = 0.2276\,\text{s} = 227.6\,\text{ms}$, while $\frac{S_{\text{act}}}{B_{\text{net}}} \leq 1.3\,\text{ms}$, establishing $\tau_{\text{crit}} > 212\,\text{ms}$ per hop. Finally, because TCP guarantees lossless ordered byte delivery of the 32-bit IEEE-754 buffer $\mathbf{h}_t \in \mathbb{R}^{4096}$ between layer slices $\mathcal{L}_k$ and $\mathcal{L}_{k+1}$, each layer l receives the exact bit pattern it receives in monolithic in-RAM execution, proving $\max_j |\mathbf{h}_{L,j}^{\text{ring}} - \mathbf{h}_{L,j}^{\text{mono}}| \equiv 0.0$. ■

Proof of Lemma 1 (Elimination of Unilateral Positional False Positives). Let $\mathcal{O} = (o_1, o_2)$ where $s(o_2) - s(o_1) = \Delta s > 0$, and suppose slot A receives additive positional bias $b_{\text{pos}} > \Delta s + \beta_A$. Under forward ordering $\mathcal{O}$, option o_1 occupies slot A and o_2 occupies slot B. By Equation (4), the calibrated logit margin between slot A and slot B is:

$$\tilde{z}_A(\mathcal{O}) - \tilde{z}_B(\mathcal{O}) = \frac{s(o_1) + b_{\text{pos}} - \beta_A - s(o_2)}{T} = \frac{b_{\text{pos}} - \beta_A - \Delta s}{T} > 0, \tag{12}$$

so $\arg\max_k p_k(\mathcal{O}) = A$, mapping to semantic choice $\Phi(\mathcal{O}) = o_1$. Because $\arg\max_k p_k(\mathcal{O}) = A$, Algorithm 1 Line 4 obligatorily triggers the inverted query $\pi(\mathcal{O}) = (o_2, o_1)$, where o_2 now occupies slot A and o_1 occupies slot B. Under $\pi(\mathcal{O})$, the calibrated logit margin is:

$$\tilde{z}_A(\pi(\mathcal{O})) - \tilde{z}_B(\pi(\mathcal{O})) = \frac{s(o_2) + b_{\text{pos}} - \beta_A - s(o_1)}{T} = \frac{\Delta s + (b_{\text{pos}} - \beta_A)}{T} > \frac{2\,\Delta s}{T} > 0, \tag{13}$$

which selects slot A in the inverted prompt, mapping to semantic choice $\Phi(\pi(\mathcal{O})) = o_2 \neq o_1 = \Phi(\mathcal{O})$. Thus $\mathbb{I}(\Phi(\mathcal{O}) = \Phi(\pi(\mathcal{O}))) = 0$, forcing $\mathcal{G}_{\text{Sys1}}(x; \mathcal{O}) = 0$ and routing the task to Tier-2. ■

Proof of Proposition 2 (Strict Monotonic Accuracy Dominance of Verifier-Gated Swarms). For any task $x \in \mathcal{T}_{\text{ver}}$, soundness of the verifier $\mathcal{V}$ guarantees that no incorrect candidate is ever accepted at Tier-1 or Tier-2 ($\Pr(\text{Correct}(h^{(k)}) = 1 \mid \mathcal{V}(h^{(k)}) = 1) = 1$). Let E_k denote the event $\mathcal{V}(h^{(k)}) = 1$ at Tier $k \in \{1, 2, 3\}$. Because Tier 2 is invoked if and only if $\neg E_1$, and Tier 3 is invoked if and only if $\neg E_1 \wedge \neg E_2$, the disjoint union $E_{\text{swarm}} = E_1 \sqcup (\neg E_1 \wedge E_2) \sqcup (\neg E_1 \wedge \neg E_2 \wedge E_3)$ has probability:

$$\Pr(E_{\text{swarm}}) = p_1 + (1 - p_1)p_2 + (1 - p_1)(1 - p_2)p_3 = 1 - (1 - p_1)(1 - p_2)(1 - p_3). \tag{14}$$

In our empirical Class 3 benchmark ($N = 8$), $p_1 = 5/8 = 0.625$ on 0.5B, and $p_3 = 3/3 = 1.00$ on the 3 residual tasks escalated to Tier-3 (`Qwen2.5-Coder-3B`), yielding $\text{Acc}(\mathcal{S}_{\text{F3}}; \text{Class 3}) = 1 - (1 - 0.625)(1 - 0)(1 - 1.0) = 1.00$ (100.0%), compared to 37.5% (3/8) for standalone Tier-1 or Tier-2. ■

B Hardware, Hyperparameter, and Calibration Configuration Details

C Complete Catalog and Granular Telemetry of the 30 Operational Tasks

To enable full independent auditing, Table 6 reports the task-by-task outcomes, ground-truth targets, resolving tiers, and exact wall-clock latencies across all 30 operational tasks (`T01` to `T30`) for all 5 policies (`lab05_tier3_closure_results.json`).

Table 5 | Complete physical hardware, hyperparameter, and neuro-symbolic gate constants for Polybee F3-Swarm v1.5 across Labs 01 to 05.

Subsystem / Layer	Parameter Symbol & Physical Description	Empirical Value / Setting	Code / Lab Reference
Host Hardware (Labs 01–05)	Processor / Microarchitecture	Intel Core i7-11370H (4C/8T @ 3.30 GHz, Tiger Lake)	`hardware_inventory_lab01.json`
	Active Vector ISA Extensions	`AVX2, AVX-512F, AVX-512DQ, AVX-512BW, AVX-512VL`	CPUID Leaf 7 Telemetry
	Total Physical RAM / Baseline Free RAM	15.788 GiB Total / 2.789 GiB Free (82% OS load)	`GlobalMemoryStatusEx`
	Unbuffered Sequential NVMe Read Speed	1,452.95–1,572.66 MB/s (`FILE_FLAG_NO_BUFFERING`)	`f3_swarm_lab02_physical_test.py`
	Dynamic CPU Power Envelope (P_{TDP})	24.0 W sustained under AVX-512 inference	Labs 03–05 Energy Integrator
Lab 02 Distributed Ring	Per-Process Hard Working-Set Quota Q_{RAM}	600.0 MiB (`QUOTA_LIMITS_HARDWS_MAX_ENABLE`)	`SetProcessWorkingSetSizeEx`
	Transformer Dimensions ($L \times d_{\text{model}}$)	24 layers $\times 4096 \times 4096$ `float32` (1.50 GiB)	3 TCP Workers (8 layers = 512 MiB/node)
Tier-1 Edge SLM (Motor A)	Model Backbone & Quantization	`Qwen2.5-0.5B-Instruct-Q4_K_M.gguf` (491 MB)	Peak RSS $\approx$ 480 MiB
	Logit Temperature T / Positional Penalty β_A	$T = 1.60$ / $\beta_A = 2.50$ ($\beta_{k \neq A} = 0.0$)	`f3_swarm_lab04_aacr_calibrated.py`
	Confidence Gate τ_{conf} / Inversion Trigger	$\tau_{\text{conf}} = 0.85$ / Trigger flip if $p_{\max} < 0.88 \vee \hat{k} = A$	Equation (5)
Tier-2 Local Escalation	Model Backbone & Quantization	`Qwen2.5-1.5B-Instruct-Q4_K_M.gguf` (1.11 GB)	Peak RSS $\approx 1,120$ MiB
	Controlled Micro-CoT Budget / Suffix Gate	$N_{\text{predict}} = 65, T = 0.0$ + 1-token `"\nResposta final: "`	`f3_swarm_lab05_tier3_closure.py`
Tier-3 Specialist & Sandbox	Model Backbone & Quantization	`Qwen2.5-Coder-3B-Instruct-Q4_K_M.gguf` (2.10 GB)	Activated on 10% of tasks (3/30)
	Rashomon Verification Criterion	4/4 (100%) Leave-One-Out in unified `exec(env, env)`	Equation (8) & (9)

Table 6 | Granular task-by-task audit matrix across all 30 real operational PT-BR tasks (`T01`–`T30`) and Policies P1 to P5.

Task ID	Class & Operational Task Description (PT-BR)	Gold Target	P1 (0.5B)	P2 (1.5B)	P3 (v0)	P4 (v1.0)	P5 (v1.5 Final)	Final Resolving Layer & Mechanism in P5	P5 Latency
T01	C1: Triagem N1 vs N2 vs Financeiro (Estorno Fatura)	C	×	✓	✓	×	✓	Tier-2 (1.5B Controlled Micro-CoT, 22 words)	2,351.8 ms
T02	C1: Detecção de Risco PIX Noturno (R$ 9.850)	A	✓	✓	✓	✓	✓	Tier-2 (1.5B System-1 1-Token Prefix)	1,099.8 ms
T03	C1: Classificação Urgência Incidente TI Gateway	D	×	✓	×	×	×(C)	Tier-2 (1.5B Micro-CoT + Suffix Closure)	2,489.3 ms
T04	C1: Análise de Sentimento SAC (Elogio Atendente)	A	✓	✓	✓	✓	✓	Tier-2 (1.5B System-1 1-Token Prefix)	1,065.9 ms
T05	C1: Risco de Churn B2B (Redução 80% Licenças)	B	✓	×	×	✓	✓	Tier-2 (1.5B System-1 1-Token Prefix)	1,124.9 ms
T06	C1: Elegibilidade Crédito Rápido c/ Restrição Serasa	B	×	✓	×	✓	✓	Tier-2 (1.5B System-1 1-Token Prefix)	1,182.4 ms
T07	C1: Detecção de Phishing / Golpe WhatsApp	A	×	✓	✓	✓	✓	Tier-2 (1.5B System-1 1-Token Prefix)	1,167.7 ms
T08	C1: Roteamento Notificação Extrajudicial Procon	A	✓	×	✓	×	✓	Tier-2 (1.5B Controlled Micro-CoT, 11 words)	1,595.7 ms
T09	C1: Score de Prioridade Atendimento VIP Tier-1	E	×	×	×	×	✓	Tier-2 (1.5B Micro-CoT + Suffix $\Psi_{\text{sem}}(5) \mapsto$ E)	2,555.5 ms
T10	C1: Intenção de Cancelamento vs Alteração Vencimento	B	×	✓	×	✓	✓	Tier-2 (1.5B System-1 1-Token Prefix)	786.0 ms
T11	C1: Detecção Discrepância Fiscal ISS (LC 116/2003)	B	×	✓	×	✓	✓	Tier-2 (1.5B System-1 1-Token Prefix)	1,130.6 ms
T12	C1: Triagem Bug Crítico vs Melhoria Padding 2px	B	×	✓	✓	✓	✓	**Tier-1 (0.5B Calibrated System-1, $p_{\max} = 99.9\%$)**	**213.2 ms**
T13	C2: Fatura Cloud Computing AWS (R$ 14.850,20)	14850.2	×	✓	✓	✓	✓	**Tier-1 (0.5B + Motor B Symbolic Normalizer)**	1,991.5 ms
T14	C2: Recibo Honorários Advocatícios (R$ 7.500,00)	7500.0	×	✓	✓	✓	✓	**Tier-1 (0.5B + Motor B Symbolic Normalizer)**	1,745.1 ms
T15	C2: Aquisição Servidores Dell (R$ 32.400,00)	32400.0	×	✓	✓	✓	✓	**Tier-1 (0.5B + Motor B Symbolic Normalizer)**	1,787.1 ms
T16	C2: Reembolso Viagem Corporativa (R$ 1.840,50)	1840.5	✓	✓	✓	✓	✓	**Tier-1 (0.5B + Motor B Symbolic Normalizer)**	1,998.8 ms
T17	C2: Licenciamento Oracle Brasil (R$ 56.000,00)	56000.0	×	✓	✓	✓	✓	**Tier-1 (0.5B + Motor B Symbolic Normalizer)**	2,349.6 ms
T18	C2: Locação Imóvel Faria Lima (R$ 18.200,00)	18200.0	✓	✓	✓	✓	✓	**Tier-1 (0.5B + Motor B Symbolic Normalizer)**	1,531.2 ms
T19	C2: Segurança e Monitoramento (R$ 4.350,00)	4350.0	✓	✓	✓	✓	✓	**Tier-1 (0.5B + Motor B Symbolic Normalizer)**	2,616.7 ms
T20	C2: Auditoria Contábil PriceAudit (R$ 45.000,00)	45000.0	✓	✓	✓	✓	✓	**Tier-1 (0.5B + Motor B Symbolic Normalizer)**	2,179.6 ms
T21	C2: Treinamento In-Company (R$ 6.200,00)	6200.0	×	✓	✓	✓	✓	Tier-2 (1.5B Escalated Structured Extraction)	4,851.9 ms
T22	C2: Indenização VendaFlex (R$ 28.900,00)	28900.0	×	✓	✓	✓	✓	**Tier-1 (0.5B + Motor B Symbolic Normalizer)**	2,245.7 ms
T23	C3: Formatar Telefone BR (Fixo 10d e Celular 11d)	LOO 4/4	×	×	×	×	✓	**Tier-3 (Qwen2.5-Coder-3B + AACR Diff)**	25,183.3 ms
T24	C3: Calcular Alíquota ISS Municipal (SP/PR/MG)	LOO 4/4	✓	✓	✓	✓	✓	**Tier-1 (0.5B Rashomon LOO-CV 4/4)**	2,983.6 ms
T25	C3: Higienizar Chave PIX (Email, CPF, EVP)	LOO 4/4	×	×	×	×	✓	**Tier-3 (Qwen2.5-Coder-3B + AACR Diff)**	24,021.4 ms
T26	C3: Dígito Verificador Módulo 11 (Pesos 2..9)	LOO 4/4	×	×	×	×	✓	**Tier-3 (Qwen2.5-Coder-3B + AACR Diff)**	21,578.1 ms
T27	C3: Parsear Data BR (DD/MM/AAAA) p/ ISO 8601	LOO 4/4	×	×	×	✓	✓	**Tier-1 (0.5B Unified Sandbox LOO-CV 4/4)**	**1,593.6 ms**
T28	C3: Desconto Comercial Progressivo por Faixa	LOO 4/4	✓	✓	✓	✓	✓	**Tier-1 (0.5B Rashomon LOO-CV 4/4)**	2,304.2 ms
T29	C3: Extrair Domínio Corporativo vs Público	LOO 4/4	×	×	×	✓	✓	**Tier-1 (0.5B Unified Sandbox LOO-CV 4/4)**	**1,893.4 ms**
T30	C3: Classificar Faixa Etária ANS (0–18 a 49+)	LOO 4/4	✓	✓	✓	✓	✓	**Tier-1 (0.5B Rashomon LOO-CV 4/4)**	**1,573.2 ms**